Instrukcja 4

by Paweł Paduch (https://achilles.tu.kielce.pl/portal/author/332bd2d158a24964b2b98a7fc2879842) published 8 years ago, last modified 2 years ago

Instrukcja do laboratorium .Net WatiHandles

1. **Wstęp**
   Na dzisiejszych zajęciach będziemy mieli do czynienia z mechanizmami synchronizacyjnymi klasy WaitHandle
   - **AutoResetEvent**
   - **ManualResetEvent**
2. **Liczby pseudolosowe (1 pkt)**
   - **Pobrać zamieszczoną początkową solucję . (https://achilles.tu.kielce.pl/portal/Members/332bd2d158a24964b2b98a7fc2879842/programowanie-wspolbiezne/laboratoria/laboratorium-10/dotnet2clean.zip) Można też tworzyć projekty i wklejać poniższe kody.**
   - **Rozpakować do swojego katalogu roboczego z projektami Visual Studio**
   - **Otworzyć solucję (plik DotNet2.sln) lub z poziomu Visual Studio (Ctrl+shift+O)**
   - **W pliku Program.cs Tworzone są wątki typu "konsument" oraz jeden wątek typu "producent"**

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading;
using System.Threading.Tasks;

namespace ProducentKonsument
{
    class Program
    {

        static int licznik = 0;
        static readonly int liczbaWatkow = 10;
        static readonly int liczbaIteracji = 10;


        static void Main(string[] args)
        {
            List<Thread> watkiKonsument = new List<Thread>();
            //Random rand = new Random(); //random wspólny
            for (int i = 0;i<liczbaWatkow;i++)
            {
                var konsument = new Thread(() =>
                {
                    //lokalny random z seedem opartym na czasie
                    Random rand = new Random();
                    //lokalny random z seedem opartym na ID (ale ID za każdym urucho
                    //Random rand = new Random(Thread.CurrentThread.ManagedThreadId)
                    //Lokalny random oparty na kombinacji ID i czasu
                    //Random rand = new Random(Thread.CurrentThread.ManagedThreadId+

                    for (int j = 0; j < liczbaIteracji; j++)
                    {
                        var czasUspienia = rand.Next(1000);
                       // Console.Write($"{czasUspienia} ");
                       Console.Write("{0}",czasUspienia);
                        Thread.Sleep(czasUspienia);
                        //Console.WriteLine($"[{licznik}] ");
                        Console.WriteLine("[{0}] ",licznik);
                    }
                });
                watkiKonsument.Add(konsument);
                konsument.Start();
            }

            Thread watekProducent = new Thread(() =>
            {
                Random randPiszacy = new Random(Thread.CurrentThread.ManagedThreadId
                try
                {
                    while (true)
                    {
                        Thread.Sleep(randPiszacy.Next(100));
                        Interlocked.Increment(ref licznik);
                    }
```

```
                }
                catch (ThreadInterruptedException)
                {
                    Console.WriteLine("\nWątek piszący został zakończony");
                }
            });

            watekProducent.Start();

            foreach (var watek in watkiKonsument)
            {
                watek.Join();
            }

            watekProducent.Interrupt();
            watekProducent.Join();

            Console.WriteLine("To już jest koniec, naciśnij ENTER...");
            Console.ReadLine();
        }
    }
}
```

- Wszystkie wątki korzystają ze wspólnej zmiennej licznik. Producent zapisuje kolejne liczby konsumenci te liczby odczytują. Uruchomić program i zobaczyć jak działa. Jest tu wiele niepokojących rzeczy.
    - Po pierwsze. Generowanie liczb pseudolosowych w kilku wątkach wygląda identycznie. Zakomentuj lokalny *Random* z *seedem* opartym na czasie (domyślny konstruktor tak robi). Odkomentuj lokalny *Random* z *seedem* opartym na Id wątku.
      Można też przetestować *Random* globalny. Jakie są różnice w generowanych liczbach? Tutaj uwaga! Pomimo pozornego zadziałania globalnego randoma istnieje niebezpieczeństwo "popsucia" go przez współbieżność. Random nie jest *thread safe* i może zacząć generować same zera.
    - Po drugie brak synchronizacji powoduje dublowanie i gubienie liczb wypisywanych przez konsumentów.

3. **Zastosowanie WaitHandles** (1 pkt.)
    - Zadeklaruj dwa obiekty klasy EventWaitHandle
        - `static EventWaitHandle pisanie = new AutoResetEvent(true);`
        - `static EventWaitHandle czytanie = new AutoResetEvent(false);`
    - Użyj powyższych obiektów do zsynchronizowania pisania i odczytywania tak by klienci wypisywali kolejne unikalne liczby
4. **Użycie *ManualResetEvent*** ***(1 pkt.)***
    - *ManualResetEvent* różni się tym, że po wywołaniu *WaitOne EventWaitHandle* nie zamyka się i trzeba go zamknąć ręcznie przez Reset();
    - Zamień *EventWaitHandle* pisanie i czytanie na odpowiedniki *ManualResetEvent* dodając wywołanie metod Reset() zaraz po wywołaniu *WaitOne*().
    - Zaobserwuj działanie aplikacji.
    - Co można zrobić by użyć *ManualReset* tak by aplikacja działała jak przy *AutoReset*? Zaimplementuj swój pomysł i sprawdź działanie.

5. **Spotkania** (0,1 pkt.)
    - Utworzyć nowy projekt *Spotkania*

- Wkleić poniższy kod

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Threading;

namespace Spotkania
{
    class Program
    {
        private static readonly EventWaitHandle wh1 =  new EventWaitHandle(false, Ev
        private static readonly EventWaitHandle wh2 =  new EventWaitHandle(false, Ev
        private static readonly int liczbaIteracji = 5;
        private static Random rand = new Random();

        static void Main(string[] args)
        {
            var watek1 = new Thread(() =>
            {
                for (int i = 0; i < liczbaIteracji; i++)
                {
                    Thread.Sleep(rand.Next(2000));
                   // WaitHandle.SignalAndWait(wh1, wh2);
                    Console.WriteLine("tadam wątek 1 iteracja " + i);
                }
            });
            var watek2 = new Thread(() =>
            {
                for (int i = 0; i < liczbaIteracji; i++)
                {
                    Thread.Sleep(rand.Next(2000));
                  //  WaitHandle.SignalAndWait(wh2, wh1);
                    Console.WriteLine("tadam wątek 2 iteracja " + i);
                }
            });
            watek1.Start();
            watek2.Start();
            watek1.Join();
            watek2.Join();
            Console.WriteLine("To jest już koniec. Naciśnij ENTER...");
            Console.ReadLine();
        }
    }
}
```

- Uruchomić
- Odkomentować *SignalAndWait*
- Uruchomić ponownie

6. **WaitHandles** (1 pkt.)
   - Utworzyć nowy projekt konsolowy SynchroWait
   - Przekopiować poniższy kod

```
using System;
using System.Collections.Generic;
using System.Threading;

namespace SynchroWait
{
    class Program
    {

        static List<Thread> watki = new List<Thread>();
        static readonly int liczbaIteracji = 5;
        static readonly int liczbaWatkow = 10;
        static Random rand = new Random();

        static void Main(string[] args)
        {
            for (int i = 0; i < liczbaWatkow; i++)
            {
                var watek = new Thread((o) =>
                    {
                        int nrWatku = (int)o;
                        Thread.Sleep(rand.Next(1000));
                        //Console.WriteLine($"Wątek nr {nrWatku} wystartował");
                        Console.WriteLine("Wątek nr {0} wystartował",nrWatku);
                        for (int j = 0; j < liczbaIteracji; j++)
                        {

                           // Console.WriteLine($"Wątek nr {nrWatku} pracuje w itera
                            Console.WriteLine("Wątek nr {0} pracuje w iteracji nr {1
                            Thread.Sleep(rand.Next(1000));
                        }
                        Thread.Sleep(rand.Next(1000));
                        //Console.WriteLine($"Wątek nr {nrWatku} zakończył działanie
                        Console.WriteLine("Wątek nr {0} zakończył działanie",nrWatku
                    });
                watki.Add(watek);
                watek.Start(i);
            }

            Console.WriteLine("---------------- Doczekaliśmy się pierwszego --------




            Console.WriteLine("---------------- Doczekaliśmy się wszystkich --------

         foreach (var w in watki)
            {
                w.Join();
            }

            Console.WriteLine("To już jest koniec, naciśnij ENTER...");
            Console.ReadLine();
        }
```

```
    }
}
```

- Uruchomić
-
- Użyć mechanizmów *WaitHandle* tak by napis *Doczekaliśmy się pierwszego* wyświetlony był po zakończeniu pracy jednego z wątków (nie po zakończeniu wątku).
- Użyć mechanizmów WaitHandle tak by napis *Doczekaliśmy się wszystkich* wyświetlony był po zakończeniu pracy (koniec wątków nie jest wymagany) wszystkich wątków.

7. **Mutex, Semaphore, Lock** (0,9 pkt)
    - Dodać nowy projekt konsolowy Muteksy
    - Przekleić poniższy kod

```
using System;
using System.Collections.Generic;
using System.Threading;

namespace Muteksy
{
    class Program
    {
        static List<Thread> watki = new List<Thread>();

        static readonly int liczbaIteracji = 10000;
        static readonly int liczbaWatkow = 100;

        static int licznik = 0;

        static void Main(string[] args)
        {

            for (int i = 0; i < liczbaWatkow; i++)
            {
                var watek = new Thread((o) =>
                {
                    int nrWatku = (int)o;

                    for (int j = 0; j < liczbaIteracji; j++)
                    {
                            licznik++;
                    }

                });
                watki.Add(watek);
                watek.Start(i);
            }


            foreach (var w in watki)
            {
                w.Join();
            }
            Console.WriteLine("Licznik: {0}",licznik);
            Console.WriteLine("To już jest koniec, naciśnij ENTER...");
            Console.ReadLine();
        }
    }
}
```

- Zsynchronizować dostęp do zmiennej licznik za pomocą locka, mutexa, semafora i semafora slim.
- Za pomocą Stopwatch zmierzyć czas wykonania wszystkich wątków bez synchronizacji oraz z kolejnymi synchronizacjami lockiem, mutexem i semaforem.

8. Oczywiście pamiętamy o sprawozdaniu